

Unit 1 – Chapter 3

Software Development Life Cycles

3.0 Objectives

The objective of this part of the unit is to introduce you to the various steps required to produce and maintain good quality software. At the end of this chapter you should be able to:

- Understand the software development life cycle
- Understand various models used for software development
- Compare different models of software development

3.1 Software Development Life Cycle

Software Development Life Cycle (SDLC) is a process used by the software industry to design, develop and test high quality software. It consists of a detailed plan describing how to develop, maintain, replace and change or improve specific software.

3.2 Software Development Process and Models

A software process is a set of related activities that leads to the production of a software product. The major software development activities include gathering requirements, software design and implementation, code testing, documentation and maintenance. There are various ways in which the above activities can be done to achieve the final software. These are known as software development models. In the sections ahead we will study some of these models.

3.3 The Code-And-Fix Model



The code-and-fix model is the concept of jumping right in and building the software, only to fix any problems along the way. It is the simplest software development process and consists of two major steps: Coding and Fixing. In this model there usually is no planning or organizing before the work begins. After some initial discussions, teams starts the *Coding* process. *Fixing* usually happens along the way so the team can fix the major bugs on the way to continue the process.

During the early years of software development, the software development was a single-person task, characterized by the following:

- 1 It was a science or an engineering application.
- 2 The developer was also the user of the software.
- 3 The requirements were fully known.
- 4 The development of a software product primarily involved coding and fixing bugs, if any.

After a few years, this type of developmental model was not enough because many changes took place in the software development environment, such as:

- 1 Computers became very popular and the application domain changed to include science, business, government etc.
- 2 Developers became different from users.
- 3 Developers spent considerable time and effort to understand user requirements. Developers changed their codes several times, sometimes even after they thought they had completed the development of the software, in order to incorporate the user requirements.
- 4 Software products became so large and complex that they had to be developed by a group of people rather than a single person. This required more planning and coordination.
- 5 Because the software products were large and a group of people where developing them, there were more chances of finding issues with the product during testing and evaluation. It became important to focus on testing and maintenance of the product.
- 6 The changing requirement of the customer meant that the product had to be modified.



All these changes led to development of other SDLC models.

3.3.1 Advantages of the Code-And-Fix Model

1. It might be very convenient for small and short projects.
2. This model is often considered a time saver for software products and is a convenient model for low-budget projects.
3. This model is very common in software groups, where there are many entry level developers interested in experimentation and observation.
4. Developers sometimes enjoy this model because they receive full control over the development process.

3.3.2 Disadvantages of the Code-And-Fix Model

1. As you rush in and begin designing and developing, big problems can occur later in the project. Often times this means going back and redoing a step which costs time and money. Products created by this model are difficult to maintain and change later.
2. Due to time crunch and the lack of testing procedures, the software might have quality-related issues as well.
3. This model is not designed to work well for medium-sized and large software product teams.
4. Due to the model's experimental nature, it is also hard to estimate the delivery work when following the Code and Fix model.

3.4 The Waterfall Model

The waterfall model is a basic software development model where the life cycle of a software product is divided into a set of phases. It is called the waterfall model due to its structural/geometric similarity of the software development process with a waterfall. Winston W. Royce published an article in 1970 from which the waterfall model was drawn. It was later refined by Boehm.



3.4.1 Characteristics/Assumptions of the Waterfall Model

- A software development process consists of a number of phases in sequence.
- In this model, a phase can be started only after the completion of the previous phase.
- There is a one-way, downward flow of information and development effort from one phase to the next. This type of development model is considered to be sequential, the phases do not overlap each other.
- Work can be divided, according to phases, among different classes of specialists.
- It is possible to associate a goal for each phase and accordingly plan the deliverable of each phase.
- The output of one phase would be the input for the next phase.
- Before the output of one phase is used as the input to the next phase, it is subjected to various types of review and verification and validation testing. The test results provide a feedback upward that is used for reworking and providing the correct output.
- Generally the output from one phase is frozen and documentation is provided to the next phase. This 'frozen' product from one phase provides a reference point for the next phase and is not changed without agreement of all involved parties.
- It is possible to develop different development tools suitable to the requirements of each phase.
- It is easy to manage such software development processes because of the defined phases and specified input and deliverable from each phase.



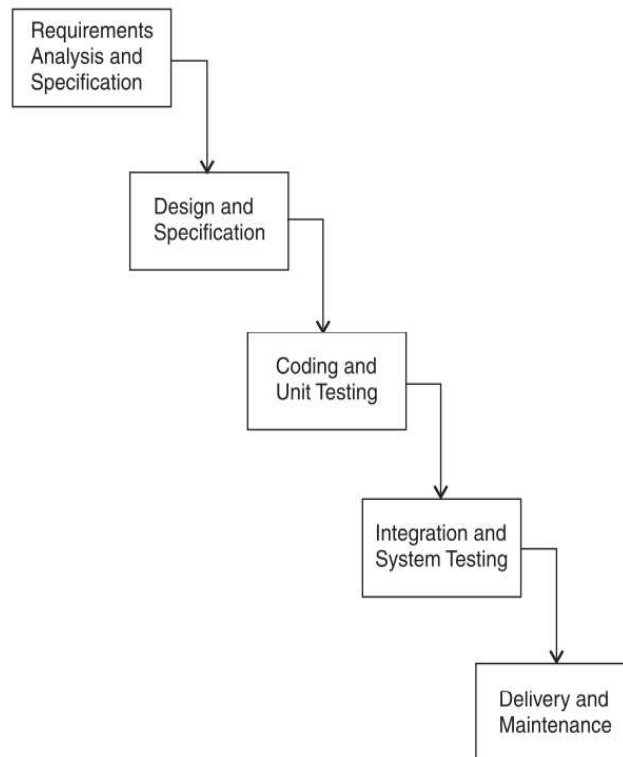


Figure 1.3.1 Waterfall Model

The Figure 1.3.1 above shows the classical waterfall model by Royce. One phase cascades to another and there is no feedback to previous phase.

The phases of software development are broadly specified as follows:

- Requirements analysis and specification: The aim of the requirement analysis and specification phase is to understand the exact requirements of the customer and document them properly.
- Design and Specification: The software requirement specification is used to come up with an overall system design.
- Coding and unit testing: During this stage, the software design is realized as a set of programs or program unit. Unit testing involves verifying that each unit meets its specification.

- **Integration and system testing:** The individual program units or programs are integrated and tested as a complete system to ensure that the software requirements have been met.
- **Delivery and maintenance:** After testing, the software system is delivered to the customer. Normally (although not necessarily), maintenance is the longest life cycle phase. The system is installed and put into practical use. Maintenance involves correcting errors and improving functionality or changing the software system.

3.4.2 Advantages of the Waterfall Model

- 1 Simple and easy to understand. All the phases are clearly defined and have specific outputs.
- 2 All phases are processed one at a time.
- 3 It is well suited for smaller software systems where requirements are well defined.
- 4 All phases are well documented. If any software team member leaves, it is easy for a new person to come in and take up the work in their place.
- 5 It encourages one to plan how components are going to interact.
- 6 It enables project managers to track progress more accurately and to uncover possible slippages early.
- 7 It reduces development and maintenance costs due to all of the above-mentioned reasons.
- 8 It enables the organization that will develop the system to be more structured and manageable.

3.4.3 Disadvantages of the Waterfall Model

- 1 The waterfall model is rigid. The results of each phase are to be frozen before the next phase can begin. It does not allow phases to overlap. In real life, phases can overlap to reduce cost and increase efficiency.
- 2 This is not a good model for complex or long ongoing software systems.
- 3 If requirements change midway, it is difficult to accommodate those in this model.



- 4 There is no working software produced till the very end of the life cycle. If any errors occur in analysis phase or the user requirements are captured improperly then these would only be known when the software is delivered.
- 5 Integration is generally done towards the end of the life cycle in this model hence it is difficult to catch issues early.
- 6 The model is heavily document driven to the point of being bureaucratic.

3.4.4 Iterative Waterfall Model

An iterative waterfall model (as shown in Figure 1.3.2) has a feedback path in the classical model. This helps with error corrections. The feedback paths allow the phases to be reworked in which errors are committed and these changes are reflected in the later phases.

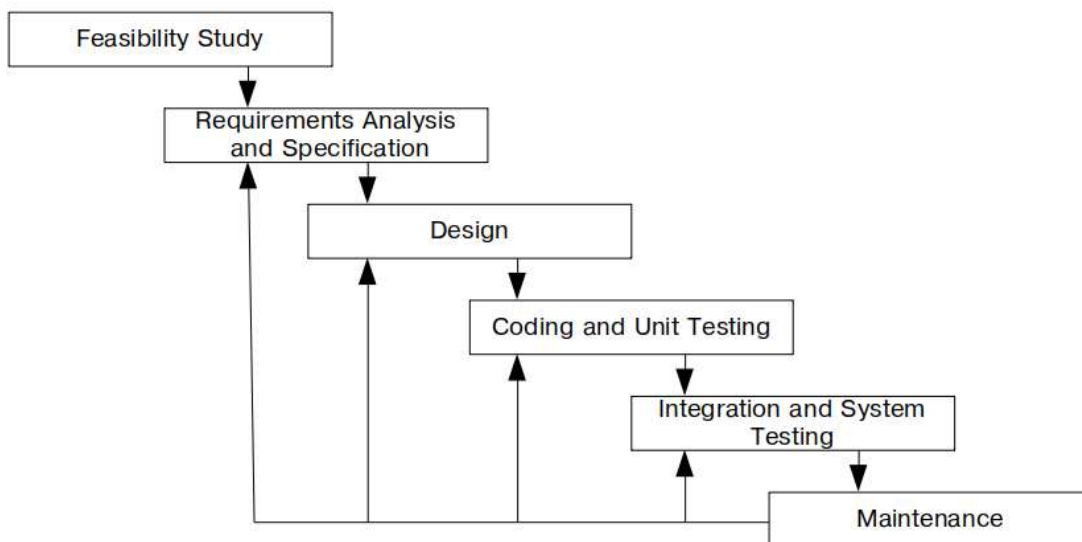


Figure 1.3.2 Iterative Waterfall Model.

3.4.5 Advantages of the Iterative Waterfall Model

1. A feedback path is provided to the previous phase so that error correction is possible.
2. Iterative waterfall model is very simple to understand and use.



3. It is highly cost-effective to change the plan or requirements in the model. Moreover, it is best suited for agile organizations.
4. In this model, less time is consumed on documenting and the team can spend more time on development and designing.

3.4.6 Disadvantages of the Iterative Waterfall Model

1. Difficult to incorporate change requests — The major drawback of the iterative waterfall model is that all the requirements must be clearly stated before starting the development phase. Customers may change requirements after some time but the iterative waterfall model does not leave any scope to incorporate change requests that are made after the development phase starts.
2. Incremental delivery not supported — In the iterative waterfall model, the full software is completely developed and tested before delivery to the customer. There is no scope for any intermediate delivery. So, customers have to wait a long for getting the software.
3. Overlapping of phases not supported — Iterative waterfall model assumes that one phase can start after completion of the previous phase, But in real projects, phases may overlap to reduce the effort and time needed to complete the project.
4. Risk handling not supported — Projects may suffer from various types of risks. But, the Iterative waterfall model has no mechanism for risk handling.
5. Limited customer interactions — Customer interaction occurs at the start of the project at the time of requirement gathering and at project completion at the time of software delivery. These fewer interactions with the customers may lead to many problems as the finally developed software may differ from the customers' actual requirements.

3.5 V-Model

A variation in the representation of the waterfall model is called the V-model. The V-model is a type of SDLC model where process executes in a sequential manner in V-shape, refer figure 1.3.3 below. It is also known as Verification and Validation model. There is a testing phase associated with each corresponding development stage. The next phase starts only after completion of the previous phase. Under the V-Model, the



corresponding testing phase of the development phase is planned in parallel. So, there are Verification phases on one side of the 'V' and Validation phases on the other side. Verification and Validation phases are joined by coding phase in V-shape. Thus it is called V-Model.

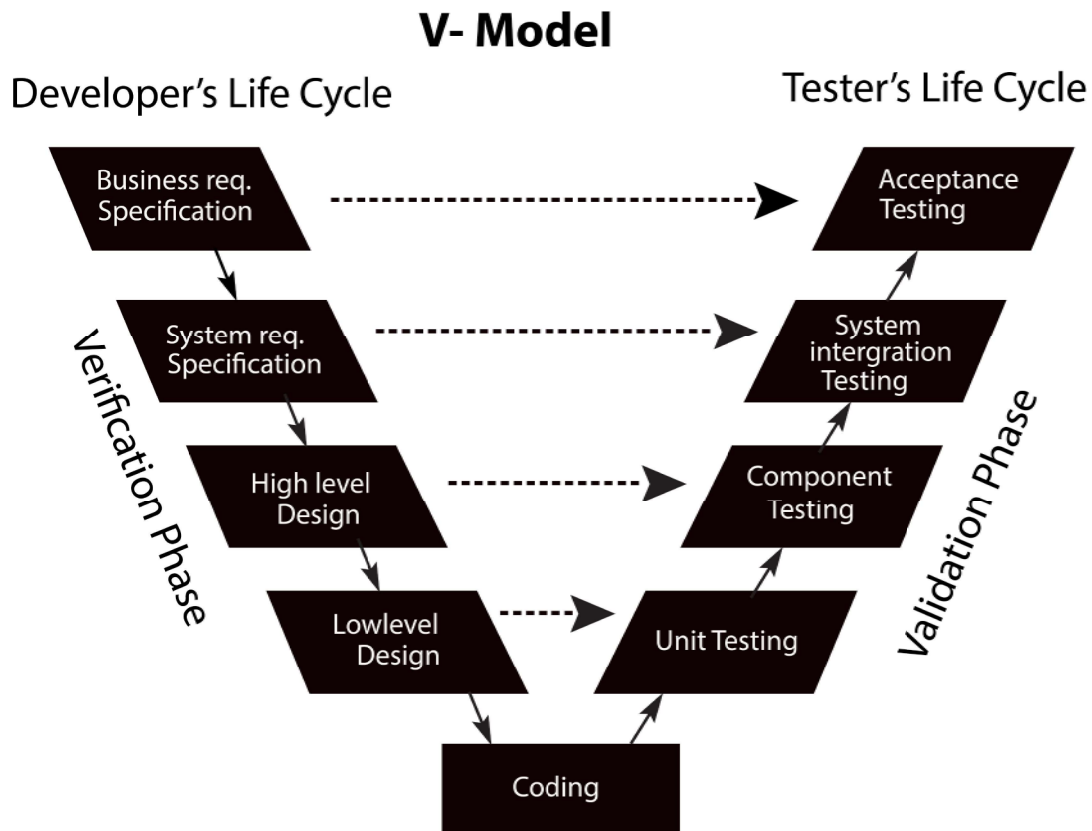


Figure 1.3.3 V-Model

Verification: It involves a static analysis method (review) done without executing code. It is the process of evaluation of the product development process to find whether specified requirements meet.

Validation: It involves dynamic analysis method (functional, non-functional), testing is done by executing code. Validation is the process to classify the software after the completion of the development process to determine whether the software meets the customer expectations and requirements.

There are the various **phases of Verification Phase** of V-model:

1. **Requirement Analysis:** This phase contains detailed communication with the customer to understand their requirements and expectations.
2. **System Design:** This phase contains the system design and the complete hardware and communication setup for developing product.
3. **Architecture Design:** The baseline in selecting the architecture is that it should understand all which typically consists of the list of modules, brief functionality of each module, their interface relationships, dependencies, database tables, architecture diagrams, technology detail, etc. This is the High-Level Design.
4. **Module or Component Design:** In the module design phase, the system breaks down into small modules. The detailed design of the modules is specified, which is known as Low-Level Design
5. **Coding Phase:** After designing, the coding phase is started. Based on the requirements, a suitable programming language is decided. There are some guidelines and standards for coding. Before checking in the repository, the final build is optimized for better performance, and the code goes through many code reviews to check the performance.

There are the various **phases of Validation Phase** of V-model:

1. **Unit Testing:** Unit Test Plans are developed during module design phase. These Unit Test Plans are executed to eliminate bugs at code or unit level.
2. **Integration Testing:** Integration Test Plans are developed during the Architectural Design Phase. In integration testing, the modules are integrated and the system is tested. This test verifies the communication of modules among themselves.
3. **System Testing:** System Tests Plans are developed during System Design Phase. System testing test the complete application with its functionality, inter dependency, and communication . It tests the functional and non-functional requirements of the developed application.
4. **Acceptance Testing:** Acceptance testing is related to the business requirement analysis part. It includes testing the software product in user atmosphere. Acceptance tests reveal the compatibility problems with the different systems, which is available within the user atmosphere. It verifies that the delivered system meets user's requirement and system is ready for use in real world.



3.5.1 Why is V-Model preferred?

1. It is easy to manage due to the rigidity of the model. Each phase of V-Model has specific deliverable and a review process.
2. Proactive defect tracking — that is defects are found at early stage.

3.5.2 When to use the V-Model?

1. Where requirements are clearly defined and fixed.
2. The V-Model is used when ample technical resources are available with technical expertise.

3.5.3 Advantages of the V-Model

1. This is a highly disciplined model and Phases are completed one at a time.
2. V-Model is used for small projects where project requirements are clear.
3. Simple and easy to understand and use.
4. This model focuses on verification and validation activities early in the life cycle thereby enhancing the probability of building an error-free and good quality product.
5. It enables project management to track progress accurately.

3.5.4 Disadvantages of the V-Model

1. High risk and uncertainty.
2. It is not a good for complex and object-oriented projects.
3. It is not suitable for projects where requirements are not clear and contains high risk of changing.
4. This model does not support iteration of phases.
5. It does not easily handle concurrent events.

3.6 The Evolutionary Model

In the Waterfall Model, the working software product is not ready till the very end of the life cycle. By contrast, in the Evolutionary Model working software products are developed constantly. These models are shown to users/customers to get their feedback and changes are made to accommodate them.

The evolutionary approach can be implemented in two forms:



- Incremental implementation.
- Prototyping.

3.7 The Incremental Implementation

In this approach a software product is developed in increments or steps. First a few functions are developed to get a working model of the software. Later more functions are added. Requirement analysis and design are done following the Waterfall Model while coding, integration, testing are done in an incremental way. Refer Figure 1.3.4 below: In this example first a Kernel was developed and in the next few increments other functionalities were added.

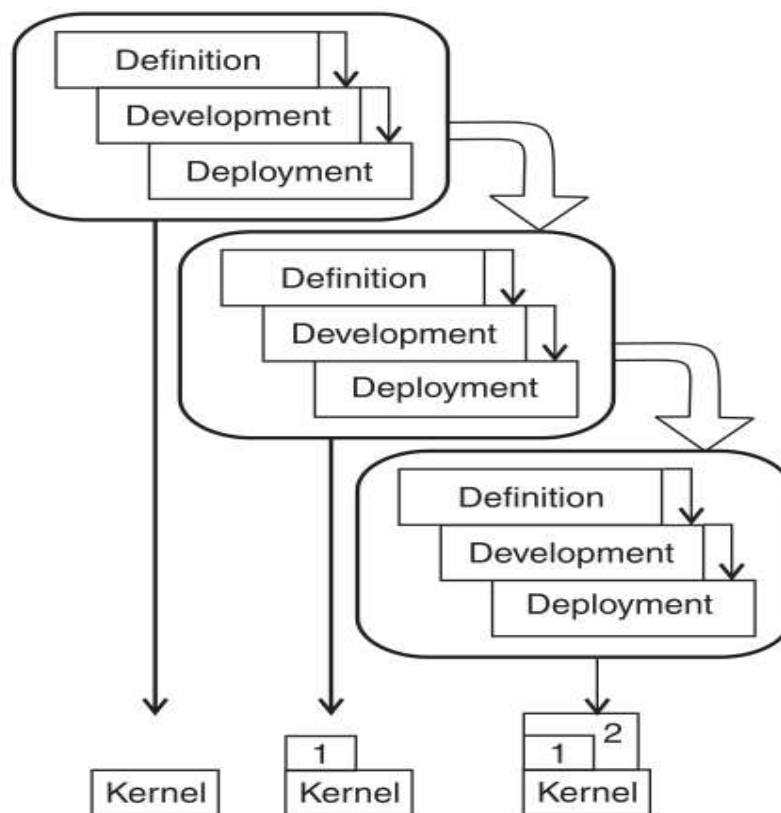


Figure 1.3.4 Incremental implementation

3.7.1 Advantages of the Incremental Implementation

- 1 Users can give their suggestions on parts of the software that can be delivered at a later point of time.



- 2 Developers concentrate on delivering the most important functionality of the software first. This ensures that these important features get the most time and attention from the developer making them more error-free.
- 3 The time taken to show some working software to the user is greatly reduced. User feedback can be incorporated into the software with ease.
- 4 Testing and error detection and error correction become easy tasks.

3.7.2 Disadvantages of the Incremental Implementation

- 1 The overall architecture or design of the software product must be done completely at the beginning of the life cycle. All increments or steps should be a part of this design.
- 2 Such kind of customer-developer contracts are not very usual.

3.8 Prototyping

In this model a working prototype of the software is given to the user for comments and feedback. It helps the user to give all their requirements in more detailed fashion.

Prototyping can be of two types:

- Throwaway prototyping (Scaffolding)
- Evolutionary prototyping

3.8.1 Throwaway Prototyping

Here the initial version of the software is developed only temporarily to get detailed requirements from the user. This is then thrown away and a second version of the software is developed using the Waterfall Model. Refer Figure 1.3.5 below:



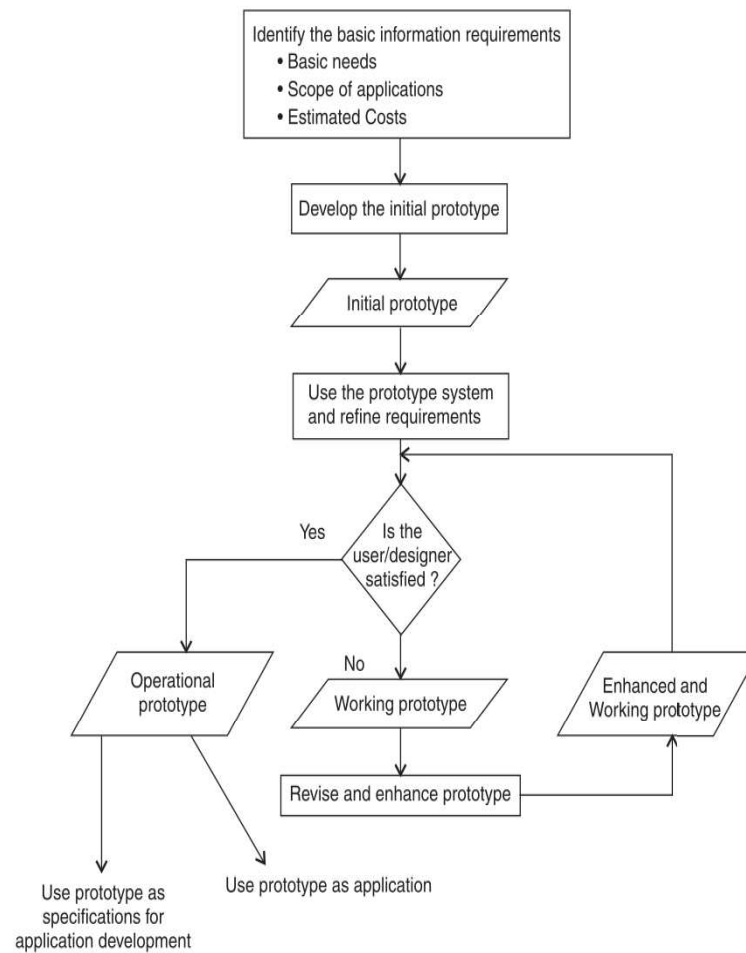


Figure 1.3.5 Throwaway prototyping

3.7.2 Evolutionary prototyping

Here the initial version is not thrown away. Changes are made to this version itself to get the final software product. Refer Figure 1.3.6 below:

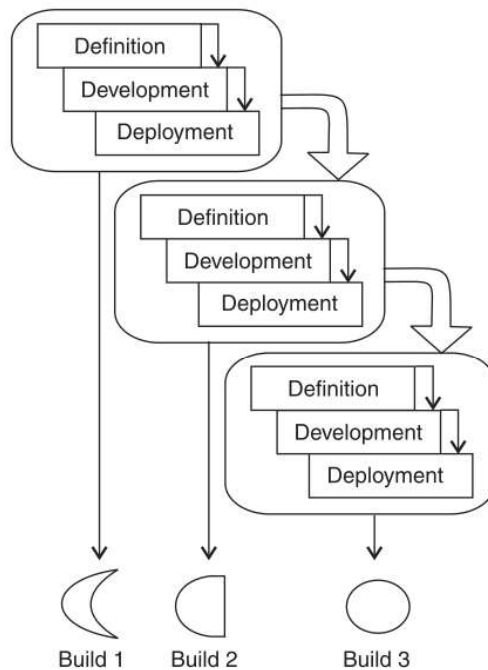


Figure 1.3.6 Evolutionary Prototyping

3.8.3 Characteristics of the Prototyping models

- Both type of prototyping assume that there is a incomplete set of requirements provided at the beginning.
- Both allow user feedback.
- An evolutionary prototype is continuously modified as the user gives their feedback. When the user is satisfied, the software product is delivered to the user/customer.
- In the throwaway prototype, the user gives their feedback to help clarify the requirements. These requirement specifications are used to then build a new piece of software following the usual phases of software development life cycle.
- There are many revisions done to the software in the evolutionary prototype resulting in bad programming structure and a difficult to maintain product.
- A throwaway prototype is usually unsuitable for testing non-functional requirements and the mode of the use of this prototype may not correspond with the actual implementation environment of the final software product.

3.8.4 Advantages of Prototyping

- 1 Misunderstanding between software developer and user may be identified.
- 2 Missing user services may be detected.
- 3 Difficult-to-use or confusing user services may be identified and refined.
- 4 The developers may find incomplete and/or inconsistent requirements.
- 5 It helps in gaining user confidence.
- 6 It helps in writing the specification.
- 7 Correct specification of requirements reduces requirements-related errors and therefore the overall development cost.
- 8 It can be used for training users before the final system is delivered.
- 9 Test cases for prototype can be used for the final software product (back-to-back testing). In case the results are the same, there is no need for any tedious manual checking.

3.8.5 Disadvantages of Prototyping

- 1 This model is costly.
- 2 It has poor documentation because of continuously changing customer requirements.
- 3 There may be too much variation in requirements.
- 4 Customers sometimes demand the actual product to be delivered soon after seeing an early prototype.
- 5 There may be sub-optimal solutions because of developers in a hurry to build prototypes.
- 6 Customers may not be satisfied or interested in the product after seeing the initial prototype.
- 7 There is uncertainty in determining the number of iterations.



- 8 There may be incomplete or inadequate problem analysis.
- 9 There may increase the complexity of the system.

3.8.6 Guidelines for developing prototypes

- The objectives of a prototype must be clear to the users. They may be to develop the user interface, validate functional requirements, or achieve a similar kind of specific objective.
- Prototyping is costly, therefore only the most important functionality should be developed. Non functional requirements can be put off till the end.
- The developers must use languages and tools that make it possible to develop a prototype fast and at a low cost.

3.9 The Spiral Model

The Spiral Model was first described by Boehm in 1988. This model integrates the characteristics of the waterfall model, the incremental implementation, and the evolutionary prototyping approach. It is also called the Meta Model.

3.9.1 Features of the Spiral Model

- The process of the software development can be depicted in the form of a spiral that moves in a clockwise direction. Refer Figure 1.3.7 below:



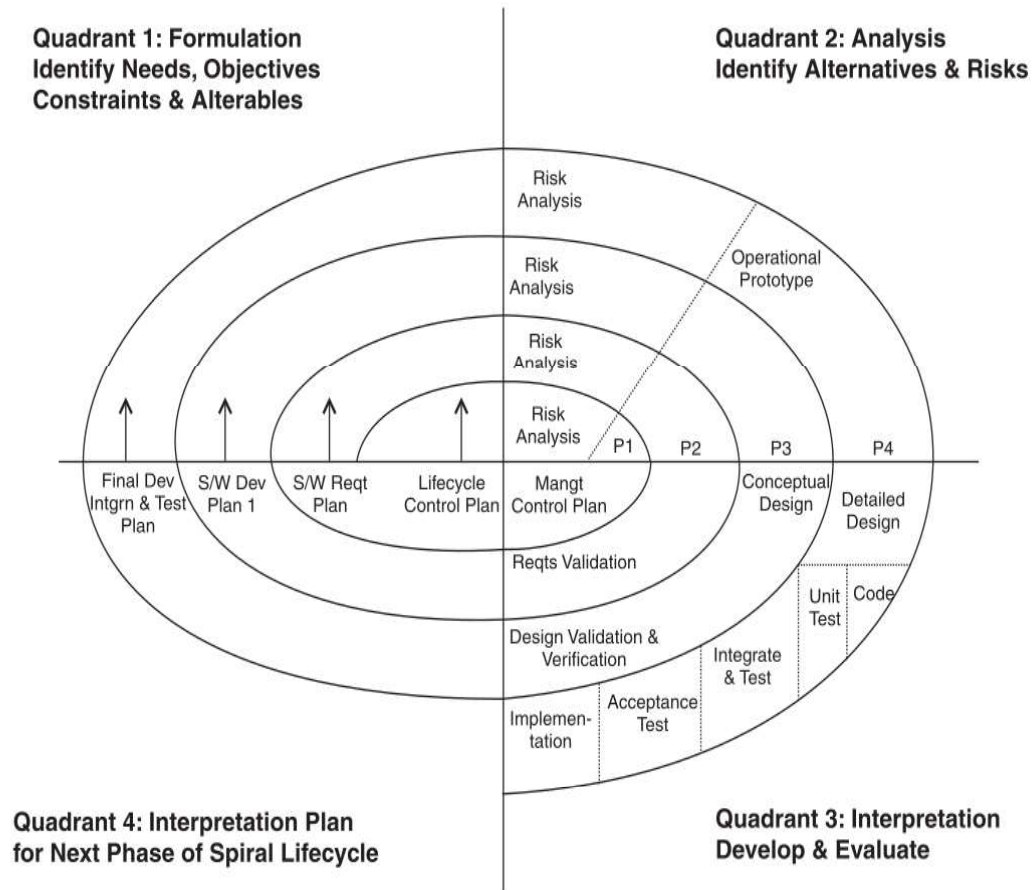


Figure 1.3.7 Spiral model

- There are no fixed phases in this model. The management can decide the required phases and these may differ for different organizations or teams. Each cycle of the spiral corresponds to a particular phase of software development. For example, the innermost cycle may deal with requirement analysis, the next cycle with design and so on.
- Each quadrant of the spiral corresponds to a particular set of activities for all phases. The four sets of activities are the following:
 - Determine objectives, alternatives and constraints: For each phase of software development, objectives are set, constraints are determined and alternate strategies are planned to meet the objectives set.
 - Identify alternatives and risks: An analysis is carried out to identify risks associated with each alternative. Prototyping is adopted to resolve them.

- Develop and evaluate: Here a model is selected for development, it could be waterfall, incremental or evolutionary prototyping. The developed software is then validated.
 - Plan next phases: The progress is reviewed and a decision is taken as to continue with the next phase or not. If the decision is to continue then plans are made for the next phase.
- The radius of the spiral represents the total cost of development; the angular dimension represents the progress; the number of cycles represents the phase of software development; and the quadrant represents the set of activities being carried out on the software development at a particular point of time.
 - An important feature of the spiral model is to assess risks. To mitigate these risks various steps like prototyping, simulation, user interviews etc. are taken.
 - The number of cycles required to develop a software depends on the risks involved. If we are developing a well understood system with clear user requirements and where risks are small, the first prototype may be accepted as the final product. Only one cycle of the spiral may suffice.

In the Figure 1.3.7, we assume that four prototypes are needed before the system requirement specifications are fulfilled. After the final result, a standard waterfall model is followed for the remaining software development life cycle phases. Thus, the spiral model represents several repetitions of the waterfall model. At each iteration, alternative approaches to software development may be followed, new functionalities may be added (the incremental implementation), or new builds may be created (prototyping).

3.9.2 Advantages of the Spiral Model

- 1 Software is produced early in the software life cycle.
- 2 Risk handling is one of important advantages of the Spiral model. It is suitable for high risk projects, where business needs may be unstable. A highly customized product can be developed using this.
- 3 Flexibility in requirements. In this model, we can easily change requirements at later phases and can be incorporated accurately. Also, additional Functionality can be added at a later date.
- 4 It is good for large and complex projects.



- 5 It is good for customer satisfaction. We can involve customers in the development of products at early phase of the software development. Also, software is produced early in the software life cycle.
- 6 Strong approval and documentation control.

3.9.3 Disadvantages of the Spiral Model

- 1 It is not suitable for small projects as it is expensive.
- 2 It is much more complex than other SDLC models.
- 3 Too much dependable on Risk Analysis and requires highly specific expertise.
- 4 Difficulty in time management. As the number of phases is unknown at the start of the project, so time estimation is very difficult.
- 5 Spiral may go on indefinitely.
- 6 End of the project may not be known early.
- 7 It is not suitable for low risk projects.
- 8 May be hard to define objective, verifiable milestones. Large numbers of intermediate stages require excessive documentation.

3.10 Software Reuse

Over time, software firms develop libraries of generic reusable software components that can be assembled to create new software products. The specification and design can be reused in addition to the code. The following components can be reused :

- The whole application system.
- Major subsystems of an application
- Modules or objects.
- Functions.



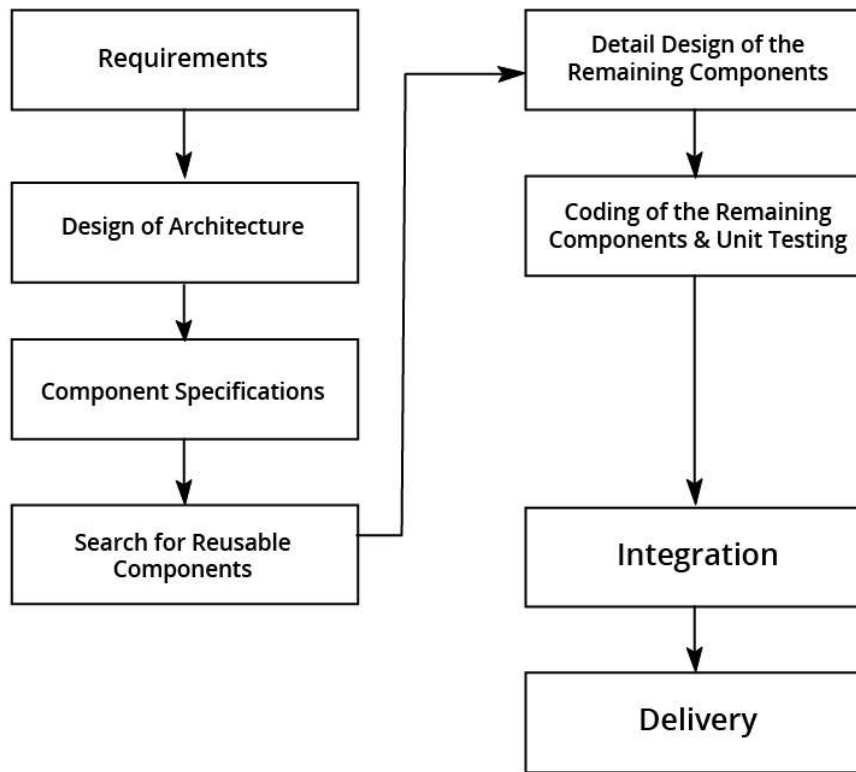


Fig 1.3.8 Steps for developing software with reusable components

The steps to developing software with reusable components are depicted in Fig 1.3.8.

3.10.1 Advantages of software reuse

- Increased system reliability.
- Reduced project risk due to less uncertain cost estimates.
- Effective use of specialists in developing generic components rather than in a wide number of projects.
- Enforcement of organizational standards in reusable components, such as common user interface and error handling procedures.
- Reduction of software development time.

3.10.2 Guidelines to be followed to make a software component reusable

1. The software component should be generalized by having general names, operations and exceptions.
2. Complete documentation should be created for a reusable component.

3. Test cases should be made available and used while integrating it with the remaining developed components.

The most common form of reuse is at the level of whole application system. The two types of difficulties are faced during this form of reuse are portability and customization.

- **Portability**

Whenever a piece of software is developed in one computer environment but is used in another environment, portability problems can be encountered. The problems with portability may be due to either of (1) Transportation or (2) Adaptation.

- *Transportation*: It involves involves physical transfer of the software and the associated data. With the widespread use of computer networking the transportation-related problems have almost disappeared today.
- *Adaptation*: It involves making the software work in a different hardware and software environment. For example the hardware of the host may have a different representation scheme (for example, a 16-bit word length) that is different from the word length of the machine where the software was developed (for example, a 32-bit word length). The operating system calls used by the software for certain facilities may not be available with the host computer operating system. Similarly, run-time and library features required by the software may not be available in a host computer.

With the advent of standards related to programming languages, operating systems, networking and user interfaces, the portability problems have reduced significantly in recent days.

- **Customization**

It has now become customary to develop generalized software packages and then customize such a package to satisfy the needs of a particular user.

3.11 Critical Comparisons of SDLC models

The five software development metrics used for comparing alternative software development life cycle models are :

- *Shortfall* — A measure of how far the software is from meeting the user requirements at any point in time.
- *Lateness* — Time delay between the appearance of a new requirement and its fulfillment.



- *Adaptability* - The rate at which a software solution can adapt to new requirements.
- *Longevity* — The time from system creation through the time it is replaced.
- *Inappropriateness* - A measure of the behavior of the shortfall over time.

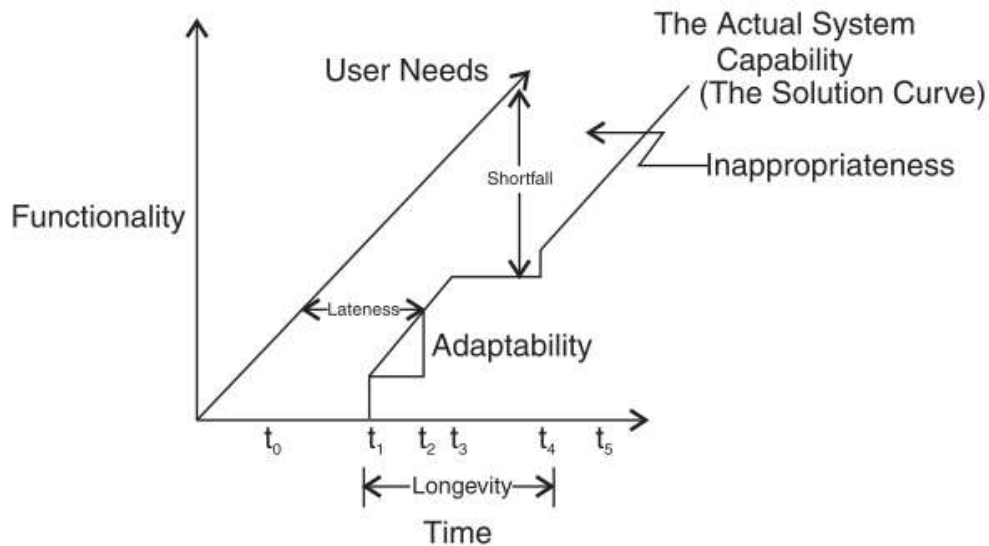


Fig 1.3.9 Software productivity metrics

Based on this framework to compare the models we can see that :

- **Waterfall model** : Looks at the life cycle of software development as being made up of sequence of phases. There is limited feedback and interactions between the phases. The waterfall model is document based.
- **Prototype model**: Allows a number of iterations between the developer and the user to receive feedback on partially built software. It is better than the waterfall model on all the 5 parameters.
- **Incremental model** : Allows addition of functionality on an initially built system to build the final system. This model is better than waterfall model and scores well on the shortness parameter.
- **Spiral model** : An incremental strategy or a prototyping strategy is followed to identify and eliminate risks and to establish user requirements and detailed software design. Coding, testing, and implementing is undertaken later in line with the waterfall model. The spiral model is risk based.
- **Software Reuse**: This model is better in terms of lateness than other models, but not in terms of shortfall or inappropriateness.

Overall it is seen that the evolution of user requirements is fundamentally ignored during conventional software development. With dynamic change in user requirements, the evolutionary prototyping result in software products that meet the user needs the best.

3.12 Case Studies

Let us see what is the most appropriate software process model that might be used as a basis for managing the development of the following systems:

1. A system to control anti-lock braking in a car
 - This is a safety-critical system that requires a lot of up-front analysis before implementation.
 - The requirements should be carefully analyzed.
 - It needs a plan-driven approach of development.

A waterfall model is therefore the most appropriate approach to use.
2. A virtual reality system to support software maintenance
 - This is a system where the requirements will change.
 - There will be an extensive user interface components.
 - The requirements cannot be predicted before the implementation and require complex programming for the software.
 - This software is not a critical system.

Incremental development with, perhaps, some user interface prototyping is the most appropriate model.

3. A university accounting system that replaces an existing system
 - This is a system whose requirements are fairly well-known due to the existing system.
 - This system will be used in an environment in conjunction with lots of other systems such as a research grant management system.

Therefore, a reuse-based approach is likely to be appropriate for this.



4. An interactive travel planning system that helps users plan journeys with the lowest environmental impact
- System would have a complex user interface.
 - This system must be stable and reliable.
 - The system requirements will change as real user experience with the system is gained.
 - Fast delivery is essential to be implemented.

An incremental development approach is the most appropriate for this. Also, the prototype model can save the time of development to focus on essential function first to be later completed.

3.13 M.C.Q.s

1. Which of the following is a valid step in SDLC framework?

A. Requirement Gathering
B. System Analysis
C. Software Design
D. All of the above
2. Which of the following is not correct model in Software Development?

A. Waterfall Model
B. P model
C. Spiral Model
D. Evolutionary model
3. The Code-And-Fix model is suitable for which of these?

A. Large and complex software products
B. Software projects with changing requirements
C. Simple software projects with few entry level developers
D. Software projects with large number of developers
4. Waterfall model is not suitable for:

A. Small projects
B. Complex projects



- C. Accommodating changes
 - D. Maintenance Projects
5. What is the major drawback of the Spiral Model?
- A. Higher amount of risk analysis
 - B. Doesn't work well for smaller projects
 - C. Additional functionalities are added later on
 - D. Strong approval and documentation control
6. Model selection is based on _____.
- A. Requirements
 - B. Development team & users
 - C. Project type & associated risk
 - D. All of the above
7. Software maintenance costs are expensive in contrast to software development
- A. True
 - B. False
8. Incremental model passes through the _____ phases each interaction
- A. Communication and modeling
 - B. Planning and construction
 - C. Deployment and maintenance
 - D. All
9. Which model assumes that the SDLC phases are sequential?
- A. Waterfall Model
 - B. Incremental Implementation
 - C. Spiral Model
 - D. Evolutionary model
10. The prototyping model of software development is
- A. The best approach to use for projects with large development teams
 - B. A risky model that rarely produces a meaningful product



- C. A useful approach when a customer cannot define requirements clearly
 - D. A reasonable approach when requirements are well defined
11. If requirements are frequently changing, which model is best suited
- A. Waterfall Model
 - B. Incremental Implementation
 - C. Spiral Model
 - D. Prototyping model
12. If requirements are understandable, easy, defined, which model is best suited
- A. Waterfall Model
 - B. Incremental Implementation
 - C. Spiral Model
 - D. Prototyping model
13. Iterative waterfall model
- A. Provides feedback path to previous phases
 - B. Never provides feedback path to previous phases
 - C. None of the above
14. Incremental implementation
- A. Phases are implemented sequentially with no feedback
 - B. Developer jumps into coding and fixing
 - C. A prototype is made of the working product
 - D. Functionality is added in increments
15. In which model is a prototype of the software product created only to be thrown away and development of the actual product using the waterfall method?
- A. Waterfall Model
 - B. Incremental Implementation
 - C. Throwaway Prototyping model
 - D. Evolutionary Prototyping model



16. In which model is the initial version of the software product not thrown away, but further enhanced to reach a final product?

- A. Waterfall Model
- B. Incremental Implementation
- C. Throwaway Prototyping model
- D. Evolutionary Prototyping model

17. The V model is also known as

- A. Verification and Validation model
- B. Incremental Implementation
- C. Throwaway Prototyping model
- D. Spiral model

18. In V-model, each and every development phase is associated with

- A. Development phase
- B. Testing phase
- C. Design phase
- D. Coding phase

3.13 Review Questions

1. Compare and contrast various traditional software process models used for software development.
2. What is a prototype model? Give its types. Under what circumstances it is beneficial to construct a prototype model? Does the construction of a prototype model always increase the overall cost of software development?
3. Why are models important in software engineering work? Do we always need them? Discuss.
4. What is Software Development Life Cycle? Explain each phase.
5. Consider two applications you are familiar with and make an argument of the pros and cons of using waterfall and incremental development processes.
6. Explain, why changes are inevitable, in complex systems. Name any two approaches used in real life projects to cope up with the change. Give an example.
7. State the differences between Verification and Validation.
8. What are the advantages of iterative approach over sequential approach?



9. What is the Waterfall model? Why is it not commonly used?
10. Explain the V-Model of software development.
11. What are the evolutionary models of software development. Explain its types.
12. What are the differences between throw-away prototyping and evolutionary prototyping?
13. Explain the incremental implementation software development model.
14. Explain the Spiral model of SDLC.
15. Explain software reuse.

3.14 MCQ Solution Key

1 D	2 B	3 C	4 C	5 B	6 D
7 A	8 D	9 A	10 C	11D	12 A
13 A	14 D	15 C	16 D	17 A	18 B

